A thinking-first local AI for coding

Research landscape, system architecture, learning and efficiency modules, and an experimental plan for a self-improving assistant on one developer's PC

Matěj Forejt
written with Claude (Anthropic)

Design study and research plan · state of the field as of 1 November 2026
Target hardware: AMD Ryzen 9 7900X · NVIDIA RTX 3080 12 GB · 32 GB DDR5 · 1 TB NVMe

This document is a design study, not a report of results. Every quantitative figure about the proposed system is a target or an order-of-magnitude estimate and is labelled as such; figures about published research are as reported by their authors.

Abstract

Current language models are next-token predictors trained once on web-scale text: they recall fluently, verify nothing, and cannot learn from use. The brain budgets its effort by novelty, learns locally and continuously, and grounds words in experience. This study asks what an individual can build today that moves toward the second shape on one consumer PC, and what research would be needed to get the rest of the way.

A survey of the field as of November 2026 finds that the two halves of the goal exist separately: explicit, inspectable reasoning in symbolic systems and small recursive networks; learning from messy experience only in large neural predictors. No system fuses them at a usable level, and the laboratories attempting it (Google's Nested Learning, LeCun's AMI Labs, Numenta, Sutton, Tenenbaum) give timelines in years. What is mature, open, and composable is a different thing: methods that make a fixed predictor learn *around* itself — evolving playbooks (ACE), verified skill libraries (Voyager), execution-scored search (LATS), expert iteration on self-verified outputs (STaR, DPO, Absolute Zero), and idle-time consolidation (sleep-time compute).

We propose an architecture that composes these into a system governed by one rule — novelty decides the compute budget — and one principle — a personal machine converts free time into quality and verified work into training signal. Known tasks resolve from a skill library in seconds; novel tasks receive the strongest reader the machine holds, plan-level search, a cheapest-first verifier cascade, and unbounded time; results drain into skills, notes, tests and a weekly LoRA, so they become known tasks. All learned state lives in plain files under version control and the system cannot edit its own verifier or benchmark. We specify model choices for the target hardware, seven multiplicative modules, a six-phase build plan with use-based acceptance tests, the measurements that would confirm or refute the design, and its ceiling: comprehension bounded by a 9B-class reader, no verifier for judgment work, and wall-clock cost on hard tasks. The missing piece — a core that learns to read and reason from little experience — remains the open research problem, and the design is shaped so that such a core would replace one component and leave the rest standing.

Contents

Abstract 2

1 Introduction 5

1.1 Motivation . . . 5

1.2 Research questions . . . 5

1.3 Contributions of this study . . . 5

1.4 Scope . . . 5

2 Background 6

2.1 How a language model is made and run . . . 6

2.2 How the brain does the same jobs . . . 6

2.3 Where the inefficiency actually lies . . . 7

3 Related work and research landscape 8

3.1 Survey . . . 9

3.2 Analysis: why no single line replaces the architecture . . . 10

4 Requirements and constraints 11

4.1 Requirements . . . 11

4.2 Hardware constraints . . . 11

5 System architecture 13

5.1 Readers and router . . . 14

5.2 The gate . . . 14

5.3 Habit tier . . . 14

5.4 Thinking tier: verified search . . . 14

5.5 Grounding: the Library . . . 15

5.6 Save tier . . . 15

5.7 Sleep . . . 15

5.8 Integrity model . . . 16

6 Learning modules 17

6.1 Context-level: the Notebook . . . 17

6.2 Skill-level: the library . . . 17

6.3 Test-level: lessons become checks . . . 17

6.4 Weight-level: expert iteration in LoRA pockets . . . 17

6.5 Harness-level . . . 18

7 Efficiency modules 19

8 Model selection for the target hardware 21

9 Evaluation protocol and build plan 22

9.1 Metrics . . . 22

9.2 Benchmark design . . . 22

9.3 Phases 22
10 Limitations and risks **24**
11 Conclusion and the research beyond it **24**
References **25**
Appendix A Core data model (sketch) **26**
Appendix B Glossary **26**

1 Introduction

1.1 Motivation

The models in daily use are extraordinary at recall and poor at the three things a working mind does cheaply: spending effort only where something is new, learning from the task just completed, and checking an answer before giving it. For a developer with a bounded domain — a Next.js and TypeScript stack, a hand-built CMS, a handful of projects — most of a frontier model's trillion-token knowledge is dead weight, and its inability to learn the developer's own idioms is a daily tax.

The question that started this study was whether the algorithm itself could be replaced: a thinking engine rather than a guessing one, efficient enough for a home computer, able to take in data and reason over it the way a person reads a manual. Section 3 reports that no such engine exists at a usable level. The remainder of the document is what can be built in its absence, designed so that the measurements it produces are themselves a contribution: a personal learning curve on real tasks, on commodity hardware, with every learned artifact inspectable.

1.2 Research questions

- **RQ1.** Can a small, local predictor be turned from a guesser into a verified reasoner on a bounded domain, and how much of the gap to frontier models does verified search close on verifiable tasks?
- **RQ2.** Can such a system improve measurably with use — on first-try pass rate, on corrections per task, on the share of work handled without deliberation — without catastrophic forgetting and without a datacenter?
- **RQ3.** How much compute does novelty-gated budgeting save relative to uniform effort, and does the fast-path share grow as the skill library grows?
- **RQ4.** Which of the published learning mechanisms (context-level, skill-level, test-level, weight-level) contributes most of the measured improvement, at what cost?

1.3 Contributions of this study

- A structured map of the 2026 research landscape, with the reasoning for why each line is or is not load-bearing for a personal system (Section 3, Figures 3–4).
- A hardware-grounded model selection for a 12 GB GPU and 32 GB of RAM, with an upgrade path (Sections 4 and 8).
- A three-tier architecture governed by novelty, with seven multiplicative modules, explicit memory, a verifier cascade and an integrity model (Sections 5–7).
- An evaluation protocol and a six-phase build plan with use-based acceptance tests (Section 9).
- An explicit statement of the ceiling, and of the research that would raise it (Sections 10–11).

1.4 Scope

Coding on the author's own stack is the domain, because it has strong verifiers (compilers, tests, builds). Design judgment, prose, and open-ended research are outside the verifiable core and are treated as assisted, not autonomous. Stripping safety training from open models is out of scope; on coding tasks model refusals are not a practical obstacle.

2 Background

2.1 How a language model is made and run



Figure 1. *The standard pipeline. Knowledge and reasoning are mixed in the same frozen weights; serving cost is fixed per token regardless of difficulty; nothing learned at use time persists.*

A transformer is trained to minimize surprise at the next token over a corpus of trillions of tokens, then aligned with human feedback and frozen. Between model generations the architecture, data selection, and post-training methods change — that is where capability jumps come from — but within a generation the weights never move. Mixture-of-experts (MoE) variants activate only a fraction of their weights per token, which reduces compute but not memory: every expert must remain reachable because the routing changes every token.

2.2 How the brain does the same jobs

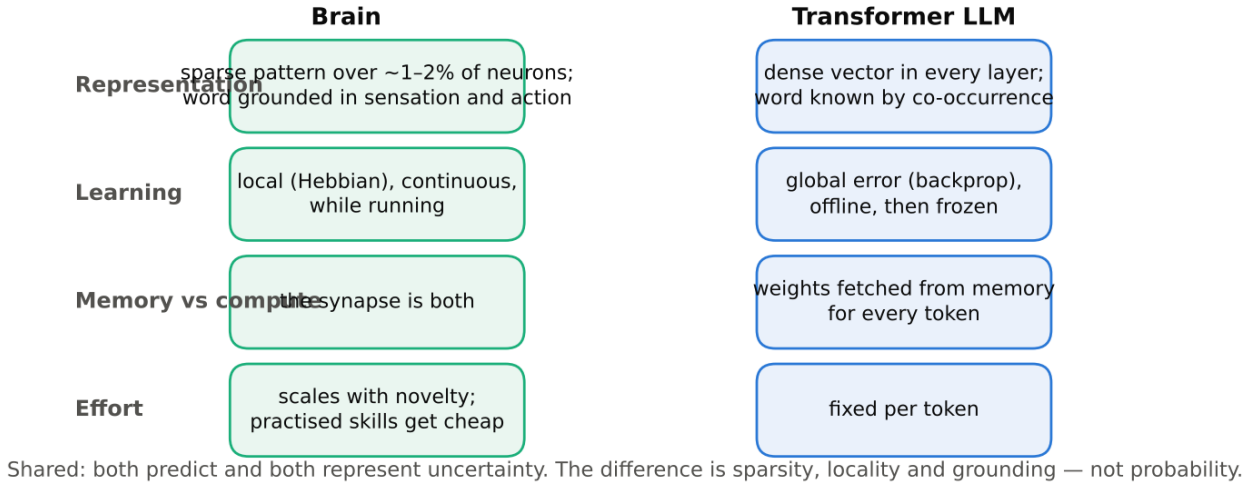


Figure 2. *Four jobs, two implementations. The brain is also a predictor that represents uncertainty; what differs is sparsity, local learning, memory fused with compute, and grounding in sensation and action.*

Recognition of a word is a competition: partially matching candidates activate and inhibit each other and the winner settles within roughly 200 ms. Meaning is a constellation across sensory, motor and emotional areas — the same neurons used to grip a hammer participate in understanding the word. Learning strengthens synapses between co-active populations locally and continuously. Practised skills migrate to cheaper circuits (basal ganglia, cerebellum) and experts show *less* cortical activity on a task than novices: effort scales with novelty, and solved

problems become compiled. Theories of thought itself — Global Workspace, Integrated Information, higher-order and predictive-processing accounts — remain unsettled; a 2025 adversarial test between the first two supported neither fully.

2.3 Where the inefficiency actually lies

It is not the probabilities. The brain is Bayesian in all the ways that matter. The waste is in four engineering choices that happen to suit GPUs: dense activation (every weight for every token, against ~1–2% in cortex); precision no one needs (16-bit weights where 4-bit loses little); memory separated from compute (every weight fetched, multiplied, written back — the dominant energy cost); and learning by a global error signal that requires stopping the network. The design in this document attacks the first and the last at the system level — sparse effort by novelty, local learning in explicit memory — and leaves the hardware questions to the neuromorphic community.

3 Related work and research landscape

Figure 3 places the relevant lines of work on two axes: how well a system learns from messy experience, and how explicit and checkable its reasoning is. The target is the upper right. Hollow markers are published but not usable for real work today.

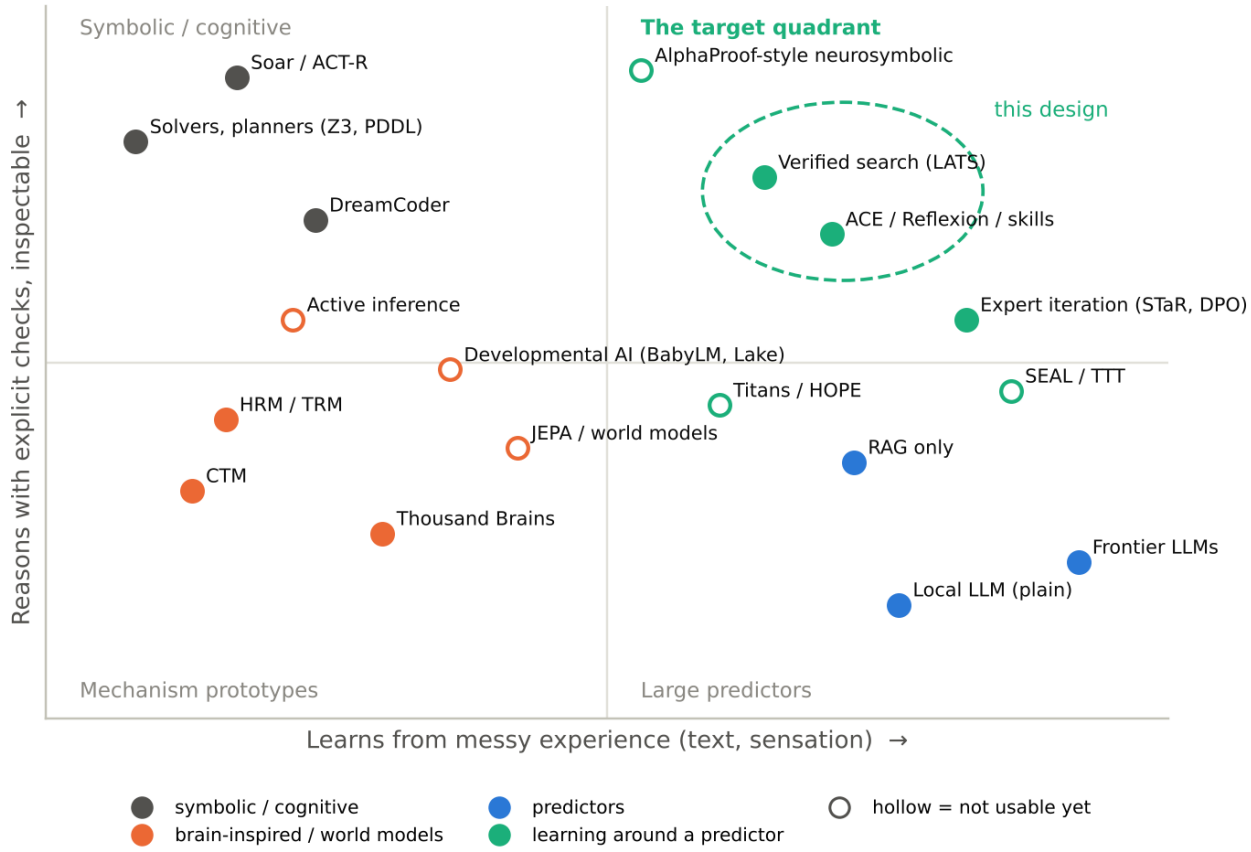


Figure 3. *Research map. The target quadrant is occupied only by methods that learn around a predictor; the brain-inspired prototypes that reason without guessing cannot yet read, and the large predictors that read cannot yet learn. This design composes the occupied quadrant and keeps the rest as experiments.*

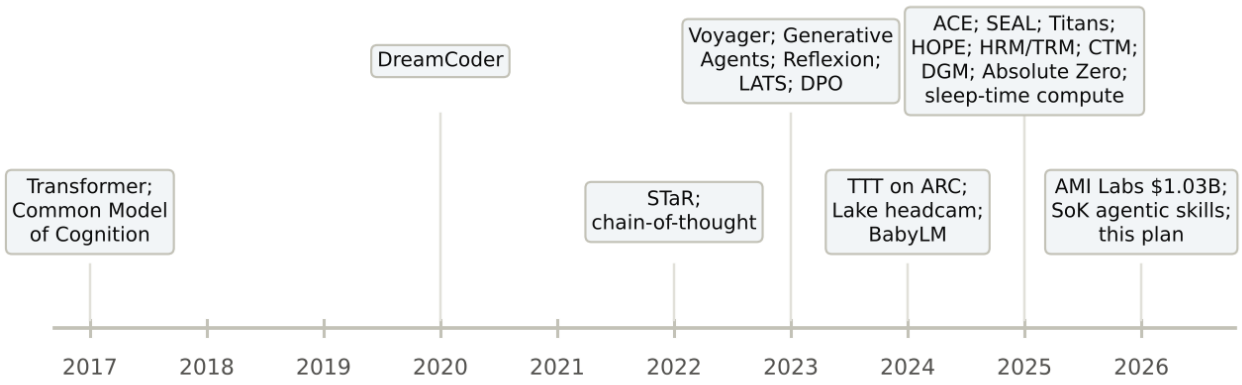


Figure 4. *Timeline of the work this design draws on. The density of 2025 reflects a field converging on three ideas from independent directions: loop on your own state, learn while working, and verify before saving.*

3.1 Survey

Line of work	Mechanism	Reported status	Role here and why
Agentic Context Engineering, ACE (ICLR 2026)	Generator / reflector / curator over an evolving playbook; delta updates; grow-and-refine. Addresses brevity bias and context collapse.	+10.6% AppWorld; with DeepSeek-V3.1 matched a top production agent; ~87% lower adaptation latency; works from execution feedback without labels.	**Core.** Defines the Notebook and the reflector/curator split.
Voyager (2023); SoK on agentic skills (2026)	Code that achieves a goal is saved as a named, composable skill.	Lifelong learning in Minecraft; library transfers across worlds.	**Core.** The Skill tier; the symbolic 'chunking' of Soar, done with a predictor.
Reflexion; Agent Workflow Memory; Dynamic Cheatsheet; A-MEM (2023–25)	Verbal lessons from failure; induced workflows; persistent strategy memory; linked atomic notes.	Consistent gains with no fine-tuning.	**Core.** Lesson notes, note linking, lessons compiled into tests.
Letta / MemGPT; sleep-time compute (2025)	Agent-managed memory blocks; a second agent consolidates while the user is idle.	Open source; runs with local models.	**Core.** Sleep and the core block.
LATS; best-of-N with unit tests (2023–)	Tree search over candidates scored by execution feedback.	Small models with sampling + tests close much of the gap to larger models on code.	**Core.** The Thinking tier.
STaR; ReST-EM; DPO; Absolute Zero (2022–25)	Generate, filter by a verifier, train on what survived; prefer verified over failed.	Absolute Zero matched models trained on tens of thousands of human examples from zero data.	**Core.** Weekly expert iteration on the developer's verified work.
Darwin Gödel Machine (2025)	Agent rewrites its own code; benchmark-scored archive; branch from any ancestor.	~20% → ~50% on a software-fixing benchmark; observed gaming its own tests.	**Idea only.** Archive + gating for harness evolution, under supervision.
Test-time training on ARC (2024); SEAL (2025)	Fine-tune per task; or self-generated study notes with RL-selected edits.	Large gains on novel puzzles; SEAL absorbs facts into weights; both forget over repeated use.	**Experiment (P6).** Too slow interactively on a 3080; weekly iteration captures most of the value.
Titans (2025); Nested Learning / HOPE (NeurIPS 2025)	Memory written by surprise during reading; several learning rates in one model.	Beat Transformers and Mamba at <1B parameters on long-context and reasoning benchmarks; community PyTorch reimplementations.	**Experiment (P6).** The most credible architecture for learning while working; not downloadable as a usable model.
HRM (2025); TRM (2025); CTM (2025)	Tiny networks looping to a fixed point; neurons encoding information in timing.	Near-perfect on hard Sudoku and mazes with 7–27M parameters; ~45% ARC-AGI-1; cannot read or code.	**Not usable.** Mechanisms to watch; no leverage on coding.
Soar; ACT-R; Common Model of Cognition (2017)	Explicit memories, impasses, chunking, production compilation.	Decades of validated results; symbols must be hand-authored.	**Not the core.** Chunking reproduced by skills; a planner may become a callable tool.

Line of work	Mechanism	Reported status	Role here and why
JEPA / V-JEPA; AMI Labs (2026); Thousand Brains (Numenta)	Predict representations, not tokens; sensorimotor reference frames.	Open (V-JEPA, Thousand Brains); AMI private, $1.03B. Pre-verbal.	**Not usable.** The real long-term bet.
BabyLM; Lake et al. (Science 2024); Tenenbaum	Child-sized data diets; word learning from one child's 61 h; probabilistic programs.	Learns nouns, not reasoning.	**Not usable.** The line that attacks the real problem.
SpikingBrain, BriLLM, Tianjic/Darwin (China, 2025)	Spiking or brain-like language models on domestic chips.	Still corpus-trained predictors; efficiency gains only.	**Not different in kind.**

Table 1. Survey of relevant work and its role in the proposed design.

3.2 Analysis: why no single line replaces the architecture

Frontier models via API are stronger at nearly everything and fail the premise: not local, not the user's, not learning from the user. They are the comparison baseline. **A plain local model** is a guesser with no verifier, search or memory — the baseline this design must beat. **Brain-inspired prototypes** cannot read; their value is scientific. **Symbolic cores** cannot absorb documents: the translation from a messy task to a formal problem is exactly the knowledge they lack. **Per-task weight updates** cost minutes of GPU per question on a 3080 and forget. **Pretraining a custom model** is three to five orders of magnitude beyond one GPU and would lose to a downloaded 9B — the 'bitter lesson' in person. **A 70B dense model at 2-bit** fits and is worse than a good 9B with search. **Multi-agent persona debates** add tokens, not truth; every measured gain in that literature traces to verification and search. **RAG alone** is grounding without reasoning or learning.

The common structure of everything that works in the target quadrant is the same: a frozen predictor, an explicit memory it reads and writes, a verifier that decides what is kept, and a loop that spends more compute when the first answer fails. That structure, not any single paper, is what Section 5 builds.

4 Requirements and constraints

4.1 Requirements

#	Requirement	Rationale
R1	Runs entirely on the target PC; no cloud dependency for reasoning or memory.	Ownership; privacy; zero marginal cost of time.
R2	Answers are verified, not merely fluent: every code change passes the cascade; every factual claim cites a retrieved span or is marked unverified.	Turns guessing into reasoning.
R3	Improves with use on the developer's own tasks, measurably, without catastrophic forgetting.	The point of the project.
R4	All learned state is explicit, inspectable and revertable.	Trust; debuggability; the user can read what it learned.
R5	Compute is spent by novelty: known tasks near-free, novel tasks as much as they need.	Efficiency; mirrors how practised skills get cheap.
R6	The system cannot weaken its own tests, verifier or benchmark.	Self-improving systems otherwise learn to look better rather than be better.
R7	Every model and store is swappable without touching the rest.	Base models improve every few months; a future learning core should drop in.
R8	Asks when unsure; never silently guesses on judgment work.	Error is tolerable; hidden error is not.

4.2 Hardware constraints

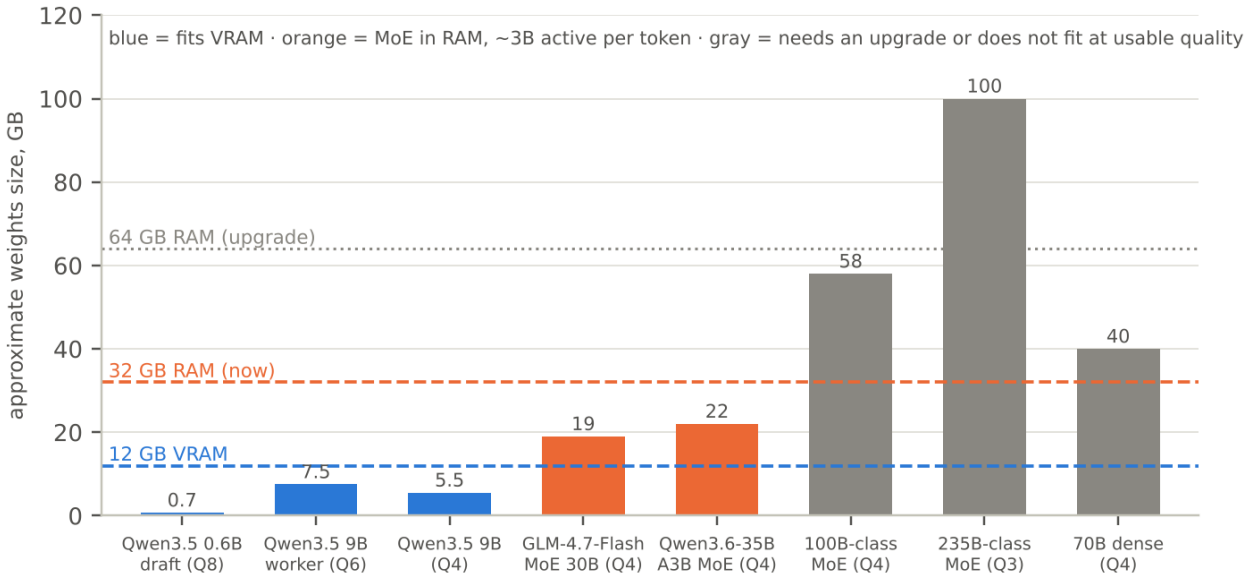


Figure 5. *What fits where. Weight sizes are approximate for the stated quantization. The 12 GB card holds a dense 9B worker with long context; the 32 GB of RAM holds a 30–35B MoE senior with ~3B active parameters per token; a 64 GB upgrade admits the 100B class; the 235B class needs 128 GB and runs slowly enough that only the Thinking tier could tolerate it.*

Constraint	Value	Consequence for the design
VRAM	12 GB	Dense worker ≤ 9–14B. KV cache quantized to q8 for long contexts. Draft model negligible.
System RAM	32 GB (board supports 128 GB)	Senior MoE ≤ 35B total at Q4 now; 100B class at 64 GB; 235B class at 128 GB, Q3, for the Thinking tier only.

Constraint	Value	Consequence for the design
Memory bandwidth	DDR5 ~80–100 GB/s; GPU ~900 GB/s	An offloaded MoE streams its ~1.5–2 GB of active weights per token from RAM: expect ~10–25 tokens/s with attention and the hottest experts pinned in VRAM. Acceptable for the slow path, not for interactive autocomplete.
A 500B-class model	~250–300 GB at Q4	Cannot be held; streamed from NVMe it yields ~1 token/s or less. MoE saves compute per token, not memory. Not attempted.
Fine-tuning	QLoRA on a 9B fits in 12 GB	Train the worker only; the senior stays frozen as teacher. Hours per weekly run.
Time	Free	The Thinking tier may run for hours; queued tasks run overnight; sleep runs when idle.

Table 2. Hardware constraints and their consequences.

Upgrade priority. 64 GB of RAM is the single purchase that changes the senior's class. A second GPU helps far less for an MoE-based design and does nothing for the RAM-bound senior.

5 System architecture

Three tiers, one gate, one save step and one sleep process, around two readers and three stores. Figure 6 gives the block view; Figure 7 the control flow of one task.

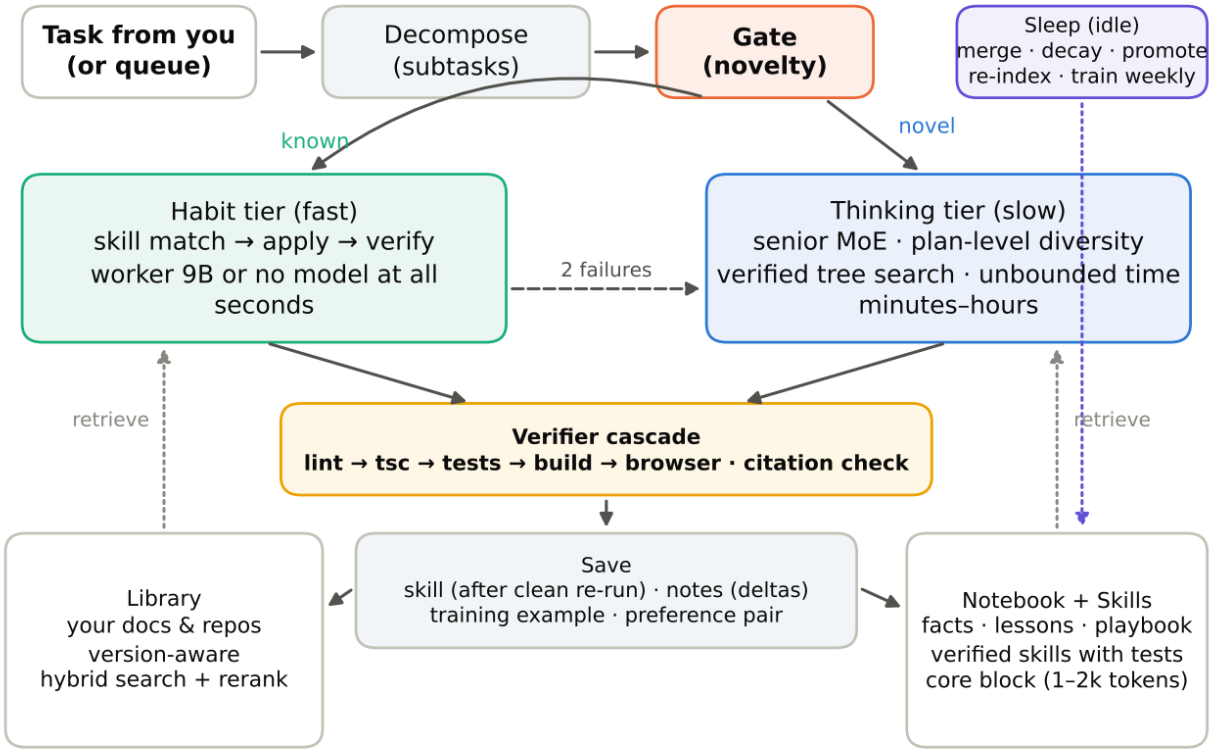


Figure 6. *System overview. The gate routes each subtask to the Habit tier (known) or the Thinking tier (novel); both pass the verifier cascade; verified results are saved to skills and notes; sleep consolidates. All stores are plain files under git.*

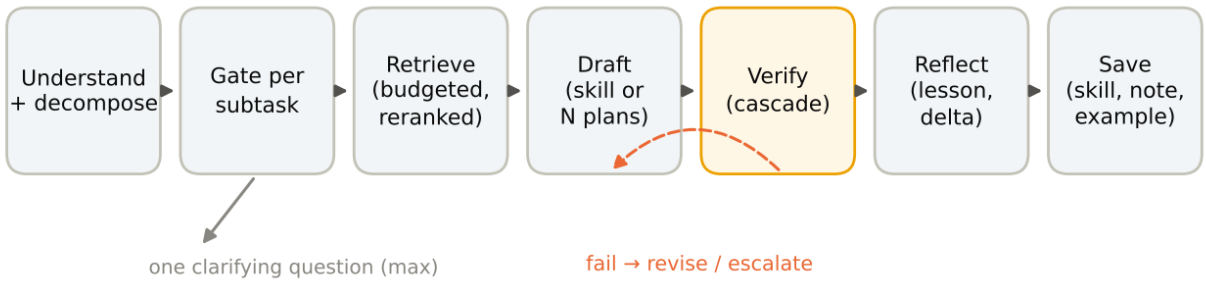


Figure 7. *Control flow of one task. One clarifying question is allowed before expensive work; verification failure revises or escalates; reflection and saving close the loop.*

5.1 Readers and router

Two language models serve two roles. The **worker** is a dense ~9B model resident in VRAM: fast, long-context, used for reading, drafting and reflecting. The **senior** is an MoE with ~3B active parameters whose experts live in RAM: slower, stronger, loaded on demand for the Thinking tier. Routing is by failure rather than by predicted difficulty: the worker tries first; two verification failures escalate to the senior. The senior is used as a *critic* of worker drafts before it is used as an *author*, since judging a draft costs a fraction of writing one. Both are served by llama.cpp behind an OpenAI-compatible HTTP interface, so the orchestration layer — written in TypeScript — never depends on a specific model.

5.2 The gate

The gate decides, per subtask, whether this is something the system has effectively done before. Three signals feed it: embedding similarity to previously solved tasks; structural footprint (which files, packages and APIs the subtask touches, against what the skill library covers); and a one-token self-estimate from the worker (known / unknown). Decisions are logged against outcomes and a small classifier refits the thresholds — the gate learns. Its errors are asymmetric: a wrong fast path costs seconds and is caught by the verifier; a wrong slow path costs an hour. The gate is therefore biased toward the fast path and relies on escalation.

Decomposition precedes gating. Novelty is local: a 'new' task is usually several old subtasks and one new one. Decomposing first and gating each part is the single largest efficiency lever in the design.

5.3 Habit tier

Skills are contracts, not snippets: declared inputs and outputs, a minimal test, a version, a success count. A matched skill is applied — often with no model call — and verified. Composition of several skills is planned by a tiny planner over their contracts. This tier is the system's equivalent of a practised motor skill: cheap, reliable, and growing with use.

5.4 Thinking tier: verified search

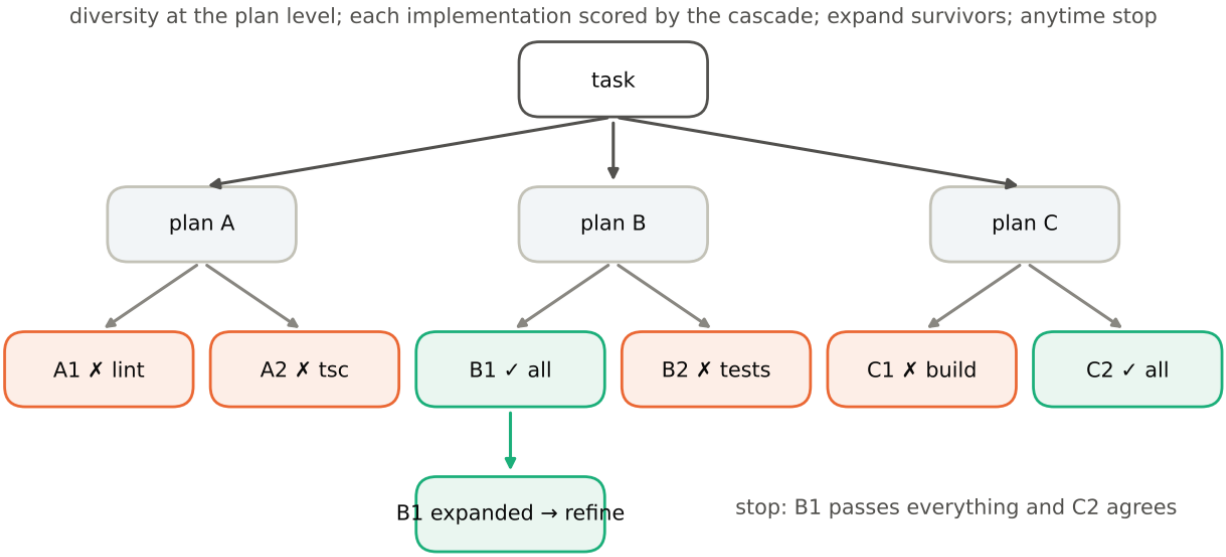


Figure 8. *Verified search. Diversity is introduced at the plan level; each implementation is scored by the cascade; survivors are expanded; search stops when a candidate passes everything and a second candidate agrees or also passes.*

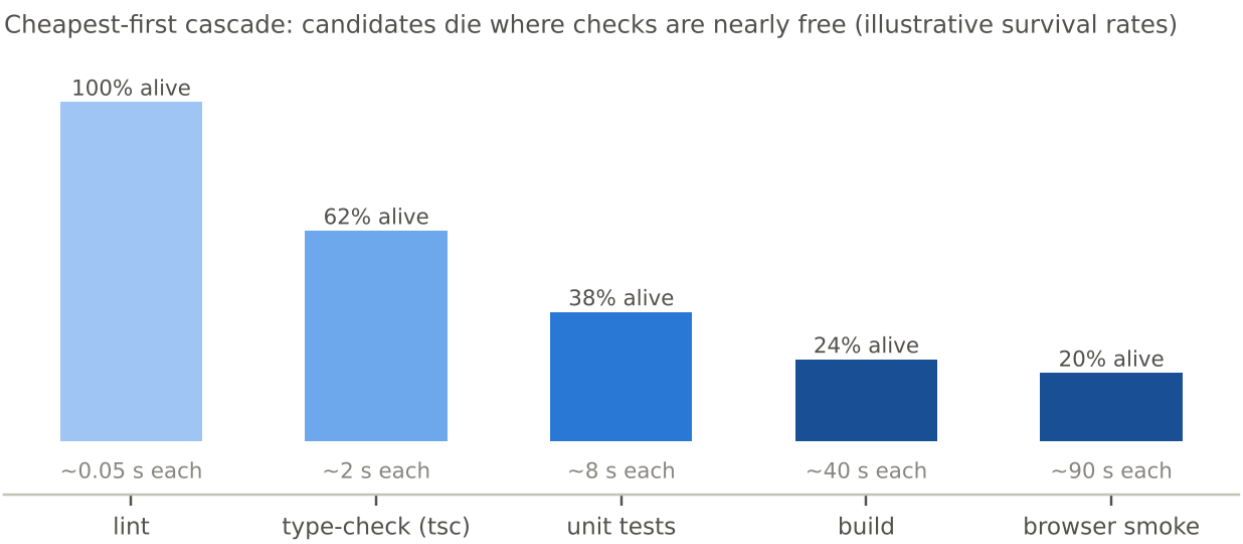


Figure 9. *The verifier cascade. Checks are ordered by cost; most candidates die where checks are nearly free, so the expensive stages see only a few.*

The Thinking tier converts wall-clock time into quality. It samples N *plans* (not N high-temperature variants of one plan), implements each with the worker, scores them with the cascade, asks the senior to critique survivors, expands the best, and repeats within a budget that scales with difficulty — one attempt for a rename, a tree of fifty for a migration, unbounded for queued overnight work. For outputs the cascade cannot judge (prose, design), it falls back to agreement among candidates plus a senior judgment, and marks the result as unverified.

5.5 Grounding: the Library

The developer's documentation and repositories are indexed with code-aware chunking (by function, component and export), an exact symbol index for lookups, hybrid keyword + embedding retrieval, and a small cross-encoder reranker. Retrieval is **version-aware**: the system reads package.json and retrieves documentation for those versions — wrong-version documentation is a leading source of confident error on a Next.js stack. A hard token budget applies because small readers degrade with long contexts: fewer, better chunks beat more. Every claim must cite a retrieved span, and every API in generated code must resolve under the type-checker.

5.6 Save tier

A verified result is saved only after it reproduces from a clean state. On save the system asks for the parameterized form of the skill and its test, deduplicates by embedding, and versions it. The reflector writes lesson and playbook deltas to the Notebook. Each success queues a training example with its reasoning trace; each success with a failed sibling queues a preference pair; each human correction queues a lesson, a test where one can be written, and a preference pair of the user's version over the system's. Corrections are the densest data the system ever sees and are treated as first-class.

5.7 Sleep

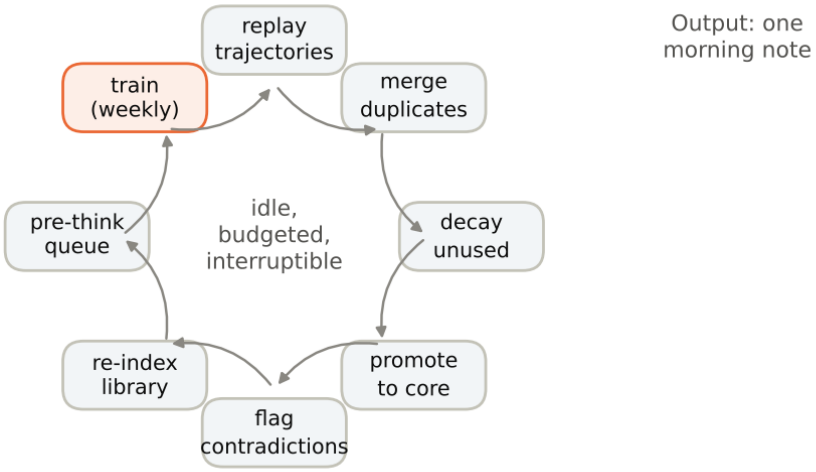


Figure 10. *The sleep cycle. Runs after an idle period, within a budget, interruptible. Training occupies the weekly slot. The output is one short note for the morning.*

5.8 Integrity model

- The system **cannot edit its own verifier, tests or benchmark**. Harness evolution proposes changes as pull requests to itself; anything touching the cascade or the test suite requires human approval. The Darwin Gödel Machine authors observed their agent faking test results; the design assumes it will happen.
- Held-out tasks are never trained on and are rotated monthly with fresh real work, so the measured curve reflects learning rather than memorization.
- Nothing enters skills or training data unverified; a skill must reproduce from a clean state before it is saved.
- Explicit memory lives in git — notebook, skills, LoRA scores, benchmark runs — diffable, revertable, inspectable.
- LoRAs are archived with scores and never merged into the base; a bad week is discarded, not absorbed.
- Contradictions and taste are the user's: the system flags, the user decides.

6 Learning modules

Learning happens at four timescales and on four substrates (Figure 10). None of it edits the base model in place, which is how the design sidesteps catastrophic forgetting while still letting the model itself move toward the developer's stack.

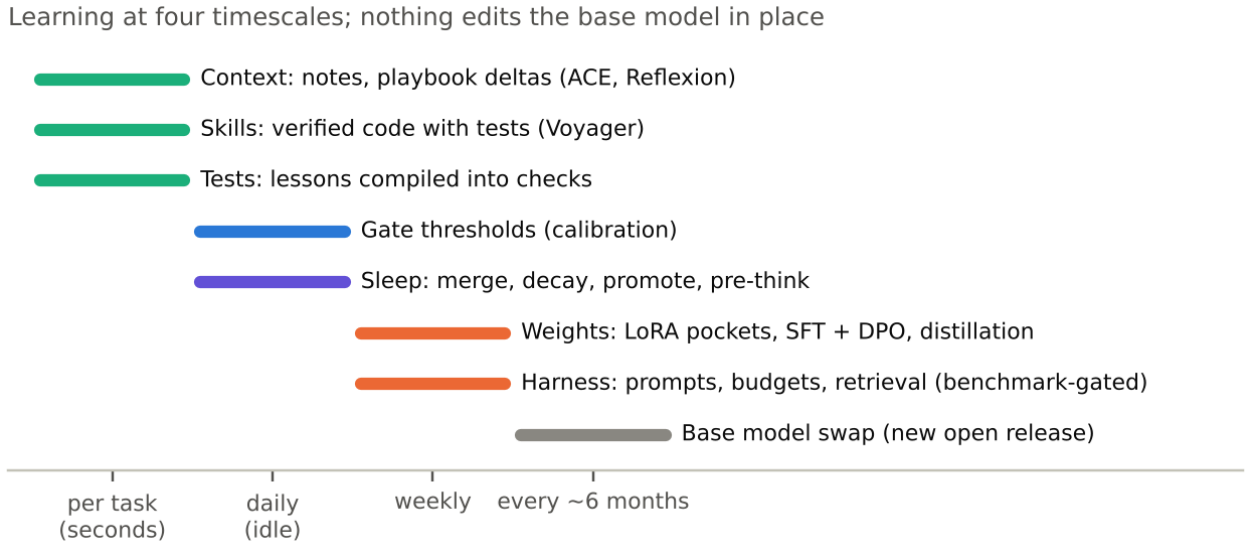


Figure 11. *Four timescales of learning. Per-task learning is in explicit context; daily consolidation in sleep; weekly learning in isolated LoRA pockets and the harness; the base model is swapped, never edited.*

6.1 Context-level: the Notebook

Following ACE, the Notebook is an evolving playbook rather than a summary. A **reflector** turns a trajectory into candidate lessons; a **curator** merges them as small, localized deltas; a grow-and-refine step adds entries and removes duplicates. Full rewrites are never performed, because rewriting loses detail (context collapse) and drifts toward generic advice (brevity bias). Notes are atomic and linked (A-MEM), typed as facts, lessons and playbook entries, and carry confidence, use-count, last-used and provenance. A small **core block** (1–2k tokens) of the highest-utility entries is present in every prompt; everything else is retrieved.

6.2 Skill-level: the library

Voyager's result — that verified code saved as composable skills produces lifelong improvement — is reproduced in a developer's domain. The symbolic ancestor is Soar's chunking: a problem solved through an impasse becomes a rule so the impasse never recurs. Skills carry tests, so they are also the fastest-verifying candidates the system has.

6.3 Test-level: lessons become checks

A lesson that explains a failure ('the build broke because the route segment was a client component') is compiled, where possible, into an assertion or test added to the cascade. Memory thereby stops being advice the model may ignore and becomes a check it cannot. This is the module expected to contribute most to quality over time, and the one with no direct precedent in the surveyed literature; it is a natural fusion of Reflexion with execution feedback.

6.4 Weight-level: expert iteration in LoRA pockets

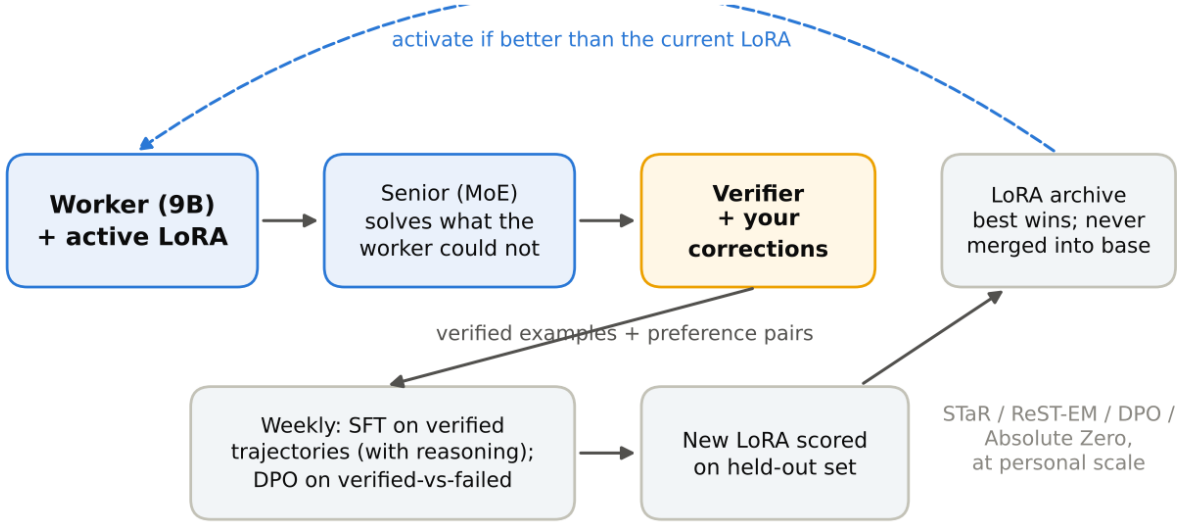


Figure 12. *Weekly expert iteration. Verified trajectories (including those only the senior could solve) and preference pairs train a LoRA on the worker; it is scored on the held-out set and archived; the best is activated. The senior is frozen and acts as teacher.*

This is STaR, ReST-EM and Absolute Zero at personal scale, with the developer's own test suite as the game's rules. Supervised fine-tuning on verified trajectories with their reasoning traces teaches the path, not only the destination; direct preference optimization on verified-versus-failed and human-corrected pairs uses data the system gets for free. One LoRA per domain is a weight-level 'pocket': forgetting is contained by isolation and a replay buffer rather than prevented by a learning rule. Distillation falls out naturally: hard tasks the senior solved become training examples for the worker, so the worker gradually does alone what needed the senior.

6.5 Harness-level

The loop proposes edits to its own prompts, retrieval settings and search budgets; the benchmark accepts or rejects them; an archive keeps every scored variant (Darwin Gödel Machine's idea, constrained by the integrity model). The gate's thresholds are refit continuously from logged outcomes.

7 Efficiency modules

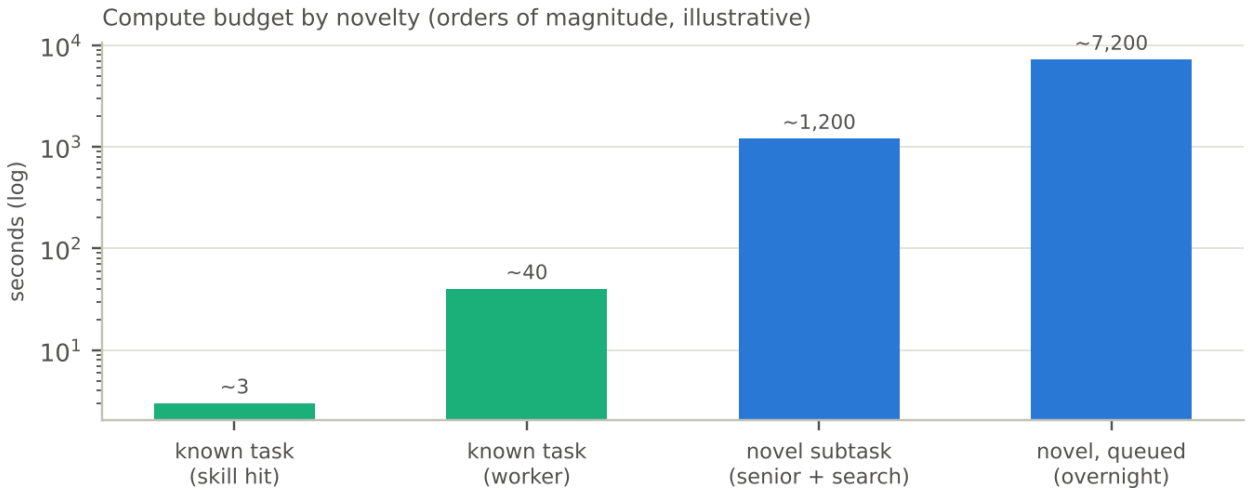


Figure 13. *Compute by novelty. The budget spans three to four orders of magnitude across the same system; the gate and the skill library determine where each subtask lands.*

Module	Effect	Basis
Decompose before gating	Most subtasks of a new task are old; only the new piece is expensive.	Hierarchical task decomposition
Asymmetric gate	Cheap errors tolerated, expensive ones avoided; escalation on failure.	Cost-sensitive classification
Cheapest-first cascade	Lint (ms) → tsc (s) → tests → build → browser (min); kill at first failure.	Cascaded classifiers
Plan-level diversity	N distinct plans beat N samples of one plan.	LATS; diverse beam
Senior as critic first	Judging costs a fraction of authoring; the senior writes only when the worker's batch fails.	Generator–verifier asymmetry
Anytime stopping	Stop at the first candidate that passes all stages when a second agrees or passes.	Self-consistency
Prefix KV caching	System prompt + core block cached across requests; the biggest fast-path latency win.	llama.cpp slot save/restore
Speculative decoding	Same-family draft model: 2–3× decoding speed, identical output.	Leviathan et al. 2023
Content-hash caches	Embeddings and verifier results keyed by hash; unchanged files are never re-checked.	Memoization
Context budget + rerank	Fewer, better chunks; small readers degrade with long contexts.	Lost-in-the-middle findings
Version-aware retrieval	Docs for the exact dependency versions in package.json.	Domain practice
Core block budget	1–2k tokens; promotion by utility (uses × success); mandatory compaction.	ACE grow-and-refine
Pre-think the queue	Retrieval and plan drafts for queued tasks during idle time.	Sleep-time compute
Presence throttling	Full power when the user is away; reduced priority when working.	Scheduling

Table 3. Efficiency modules, their effect, and the idea each rests on.

Expected order of value. The cascade, prefix caching and decomposition are plumbing and should roughly double throughput on their own. Lessons-as-tests and the corrections pipeline are where quality compounds. Pre-thinking and gate calibration pay only once the system has months of history.

8 Model selection for the target hardware

All open weights, all served by llama.cpp. Figures are as reported by vendors and third-party trackers on 1 November 2026 and should be re-checked at install time; the slot structure is what matters, the names will change.

Slot	Selection	Justification	Serving notes
Worker	Qwen3.5-9B Instruct — dense, Apache 2.0, March 2026, 262K context	Best chat-style coding in the 8–12 GB tier; long context; same family as the draft model and the MoE option B.	Q6_K (~7.5 GB); KV cache q8; resident.
Senior, option A (quality)	GLM-4.7-Flash — MoE, ~30B total / ~3B active, MIT, January 2026, 128K	SWE-bench Verified 59.2%: the strongest coding score of anything that runs on this machine.	Q4_K_M, experts in RAM, attention and hot experts in VRAM. No cross-family drafting.
Senior, option B (speed-coherent)	Qwen3.6-35B-A3B — MoE, 35B total / 3B active, Apache 2.0	Same tokenizer as the worker: the worker drafts for it (2–3× faster). Slightly weaker reported coding scores than A.	UD-Q4 (~22 GB) with KV offload, or Q3 (~17 GB). Benchmark A vs B on the 20 tasks; keep the winner.
Draft	Qwen3.5 0.6B–1.7B	Speculative decoding for the worker.	Negligible VRAM; must share the target's tokenizer.
Autocomplete (optional)	Qwen2.5-Coder 7B	Best local fill-in-the-middle if an editor plugin is added.	Loaded on demand; cannot co-reside with the worker on 12 GB.
Embeddings	Qwen3-Embedding 0.6B (alt. nomic-embed-code)	Strong on code and multilingual text; tiny.	llama.cpp embedding endpoint; hash-cached.
Reranker	Qwen3-Reranker 0.6B (alt. bge-reranker-v2-m3)	Precision before the prompt.	CPU acceptable.
Fine-tuning stack	Unsloth or LLaMA-Factory (QLoRA); TRL (DPO)	QLoRA on a 9B fits in 12 GB with a 4-bit base and gradient checkpointing.	Worker only; weekly; LoRA archive.

Table 4. Model selection. The structure (worker / senior / draft / embed / rerank / tune) is stable; the names are 2026's best fits.

9 Evaluation protocol and build plan

9.1 Metrics

Metric	Definition	Answers
First-try pass rate	Share of benchmark tasks whose first returned solution passes the full cascade.	RQ1, RQ2
Corrections per task	Human edits required after the system's final answer.	RQ2
Fast-path share	Share of subtasks resolved by the Habit tier without escalation.	RQ3
Cost per task	Wall-clock seconds, tokens, and watt-hours (measured at the wall).	RQ3
Held-out delta	Pass-rate change on the 10 rotated held-out tasks, per LoRA and per harness change.	RQ2, RQ4
Ablations	Each learning module disabled in turn on the benchmark.	RQ4

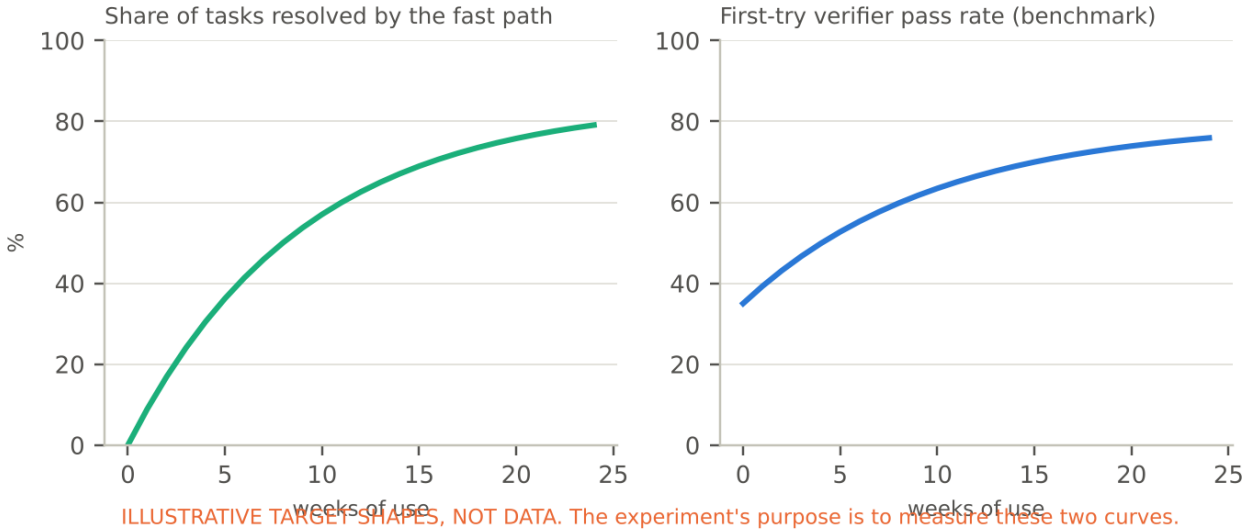


Figure 14. *Target shapes for the two headline curves. These are not data; the experiment exists to measure them. A flat fast-path share means the skill library or the gate is wrong; a flat pass rate means the memory design is wrong.*

9.2 Benchmark design

Twenty real tasks from the developer's own projects, written as acceptance tests (a failing test or build that must pass), spanning routine (rename, add a field), moderate (new component with state, data migration) and hard (cross-cutting refactor, performance fix). Ten further tasks are held out, never used for training or harness tuning, and replaced monthly with fresh work. The whole suite runs overnight on every proposed LoRA or harness change and monthly as a matter of course.

9.3 Phases

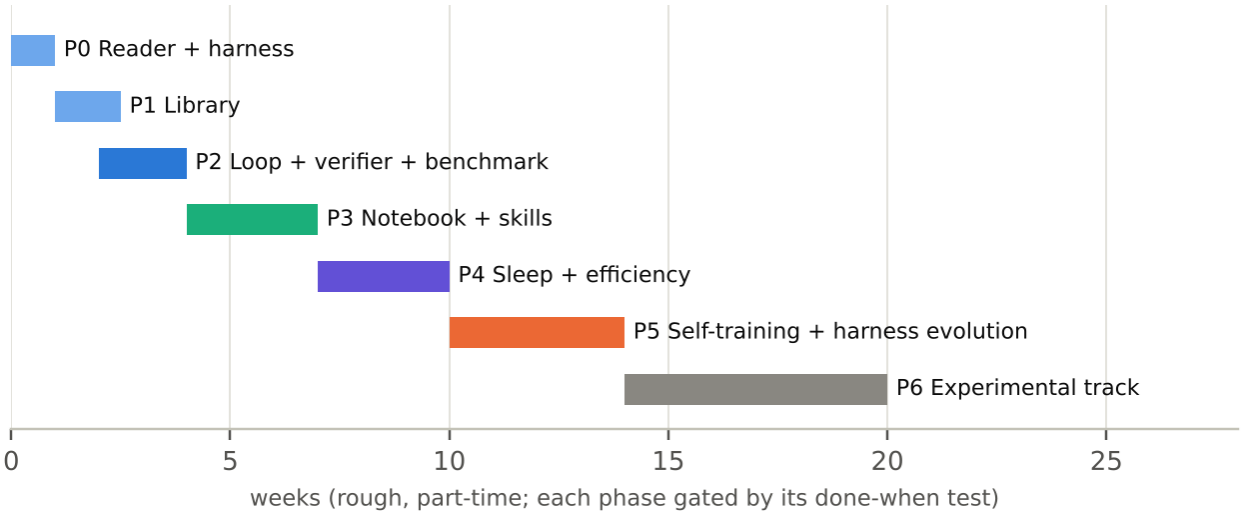


Figure 15. *Build plan. Durations are rough, part-time estimates; each phase is gated by its acceptance test, not by the calendar.*

#	Phase	Done when	Who
P0	**Reader + harness.** llama.cpp servers; TypeScript CLI that sends a task, runs the cascade, shows the result.	It fixes a real TypeScript error in one of your repos and the build passes.	Assisted
P1	**Library.** Ingest docs and repos; version-aware, code-aware chunks; hybrid search + rerank; symbol index; citations.	Ten questions about your own CMS answered with the right file cited; zero invented APIs.	Assisted
P2	**Loop + verifier + benchmark.** Decompose, rule-based gate, retrieve, draft, verify, revise; ask-when-unsure; senior as critic; 20 + 10 tasks recorded.	Baseline numbers exist for every metric in 9.1.	Loop hand-written
P3	**Notebook + skills.** Reflector/curator deltas; lessons; skill contracts; reproduce-before-save; lessons-as-tests; corrections pipeline.	Benchmark re-run: pass rate up, corrections down, fast-path share measurable. Otherwise fix the memory design first.	Notebook model hand-written
P4	**Sleep + efficiency.** Idle consolidation; core block; pre-think; prefix caching; speculative decoding; senior A vs B.	After a month: notebook smaller than the raw log; curve still rising; fast path handling most routine work.	Sleep hand-written
P5	**Self-training + harness evolution.** Weekly SFT + DPO QLoRA; LoRA archive; distillation; gate calibration; gated harness edits.	A trained LoRA beats the untrained worker on the held-out set at equal cost — or does not, and you know why.	Training assisted; gating hand-written
P6	**Experimental track.** (a) Titans/HOPE-style neural memory from open reimplementations trained on your trajectories; (b) per-task test-time training for the hardest queued tasks; (c) a symbolic planner as a callable tool.	Any one beats the P5 system on the benchmark at equal cost — or is documented as not worth it.	Research; assisted

Table 5. Phases and acceptance tests. 'Hand-written' means closed-book, no agent; 'assisted' means Claude drafts and the author reviews.

10 Limitations and risks

- **Comprehension cap.** If none of N candidates is right, search cannot fix it. The senior and 'ask the user' are the fallbacks; a better small reader every six months is the upgrade path. This is the ceiling of the whole design, and it is the frontier the research in Section 3 is circling.
- **Unverifiable work.** Design, UX and architectural judgment have no verifier; there the system is a competent assistant that defers, not a thinker.
- **Overfitting.** Self-training can tune to the benchmark; the rotating held-out set and the LoRA archive are the defenses. A few off-domain tasks should be tracked to detect erosion of general ability.
- **Forgetting.** LoRA isolation and replay limit it; they do not eliminate it. The base model is never edited.
- **Wall-clock.** A hard task may take an hour. This is the deal: time for quality, and only for novel work.
- **Goodhart.** A system that can edit its own tests will. The integrity model (5.8) exists for this reason and must be implemented before P5.
- **Measurement validity.** Twenty tasks is a small benchmark; month-to-month noise will be real. Report confidence intervals and prefer trends over single runs.
- **Model drift.** Named models will be superseded within months; the plan is written against slots, not names.

11 Conclusion and the research beyond it

The study set out to find a thinking algorithm rather than a guessing one, efficient enough for a home computer, able to learn what it is given and reason with it. The field does not yet contain that algorithm. It does contain, published and open, every piece needed to make a fixed predictor behave as if it were learning: explicit memory with delta updates, verified skill libraries, execution-scored search, expert iteration on self-verified work, and idle-time consolidation. Composed under one rule — spend by novelty — and one principle — convert free time into quality and verified work into training signal — those pieces form a system that no downloadable tool provides, because every one of them needs a verifier, a benchmark and time, which products cannot assume and an individual has.

What the experiment can establish is modest and real: whether a personal learning curve exists on commodity hardware, which mechanism drives it, and at what cost. What it cannot establish is the thing the author asked for first — a core that learns to read and reason from a little experience. That remains the open problem of the decade, pursued by Nested Learning and Titans on the architecture side, by world models and developmental AI on the learning side, and by cognitive architectures on the reasoning side. The design here is shaped so that when such a core arrives, it replaces the reader, and the library, the notebook, the skills, the verifier and the benchmark keep working around it.

References

Links are given where they were checked during preparation; other entries are cited by title and year.

- Zhang, Q. et al. *Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models.* ICLR 2026. arxiv.org/abs/2510.04618
- Zweiger, A. et al. *Self-Adapting Language Models (SEAL).* MIT, 2025. arxiv.org/abs/2506.10943
- Behrouz, A. et al. *Nested Learning: The Illusion of Deep Learning Architectures (HOPE).* NeurIPS 2025. research.google/blog/introducing-nested-learning-a-new-ml-paradigm-for-continual-learning/
- Behrouz, A., Zhong, P., Mirrokni, V. *Titans: Learning to Memorize at Test Time.* 2025. Community implementation: github.com/lucidrains/titans-pytorch
- Wang, G. et al. *Hierarchical Reasoning Model.* 2025. arxiv.org/abs/2506.21734
- Jolicoeur-Martineau, A. *Less is More: Recursive Reasoning with Tiny Networks (TRM).* Samsung SAIL Montréal, 2025.
- Darlow, L. et al. *Continuous Thought Machines.* Sakana AI, 2025.
- Zhang, J. et al. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.* Sakana AI / UBC, 2025.
- Wang, G. et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models.* 2023. arxiv.org/abs/2305.16291
- *SoK: Agentic Skills — Beyond Tool Use in LLM Agents.* 2026. arxiv.org/abs/2602.20867
- Park, J. S. et al. *Generative Agents: Interactive Simulacra of Human Behavior.* 2023.
- Shinn, N. et al. *Reflexion: Language Agents with Verbal Reinforcement Learning.* 2023.
- Zhou, A. et al. *Language Agent Tree Search Unifies Reasoning, Acting and Planning in Language Models (LATS).* 2023.
- Wang, Z. et al. *Agent Workflow Memory.* 2024. · Suzgun, M. et al. *Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory.* 2025. · Xu, W. et al. *A-MEM: Agentic Memory for LLM Agents.* 2025.
- Packer, C. et al. *MemGPT / Letta*; Lin, K. et al. *Sleep-time Compute: Beyond Inference Scaling at Test-time.* 2025. forum.letta.com/t/sleeptime-agents-for-memory-consolidation-best-practices-guide/154
- Zelikman, E. et al. *STaR: Self-Taught Reasoner.* 2022. · Singh, A. et al. *Beyond Human Data: Scaling Self-Training (ReST-EM).* 2023. · Rafailov, R. et al. *Direct Preference Optimization.* 2023. · Zhao, A. et al. *Absolute Zero: Reinforced Self-play Reasoning with Zero Data.* 2025.
- Akyürek, E. et al. *The Surprising Effectiveness of Test-Time Training for Abstract Reasoning.* MIT, 2024. · ARC Prize: arcprize.org
- Leviathan, Y. et al. *Fast Inference from Transformers via Speculative Decoding.* 2023.
- Thousand Brains Project (Numenta nonprofit): github.com/thousandbrainsproject/tbp.monty · AMI Labs funding, March 2026: siliconangle.com/2026/03/10/yann-lecuns-new-startup-ami-labs-raises-1-03b-train-world-models/
- Vong, W. K. et al. *Grounded language acquisition through the eyes and ears of a single child.* Science, 2024. · BabyLM Challenge. · Lake, B. et al. *Building Machines That Learn and Think Like People.* 2017.
- Laird, J., Lebiere, C., Rosenbloom, P. *A Standard Model of the Mind.* AI Magazine, 2017. · Ellis, K. et al. *DreamCoder.* 2020.
- *BriLLM: Brain-inspired Large Language Model.* 2025. arxiv.org/abs/2503.11299 · *SpikingBrain.* CAS, 2025.
- Local coding models by VRAM tier (2026): insiderllm.com/guides/best-local-coding-models-2026/ · model facts: llm-stats.com
- llama.cpp (server, speculative decoding, tensor override for expert offload, KV-cache quantization, slot save); Unsloth; LLaMA-Factory; TRL.

Appendix A Core data model (sketch)

A first cut of the types the hand-written core would use. Everything is serializable to plain JSON and lives in git.

```
type Id = string;
type Note = { id: Id; kind: 'fact' | 'lesson' | 'playbook'; text: string; links: Id[];
  confidence: number; uses: number; lastUsed: number; source: { taskId: Id; trajectory: Id } };
type Skill = { id: Id; name: string; description: string; inputs: Param[]; outputs: Param[];
  code: string; test: string; version: number; successes: number; failures: number; embedding: Id
};
type Candidate = { id: Id; planId: Id; patch: string; stage: 'lint' | 'tsc' | 'tests' | 'build' |
'browser' | 'passed';
  failedAt?: string; log?: string };
type Task = { id: Id; text: string; subtasks: Subtask[]; createdAt: number; status: 'queued' |
'running' | 'done' | 'asked' };
type Subtask = { id: Id; text: string; gate: 'fast' | 'slow'; gateSignals: Record<string, number>;
  candidates: Candidate[]; result?: Id; escalations: number };
type Op = { id: Id; at: number; kind: 'note.delta' | 'skill.add' | 'skill.retire' | 'gate.log' |
'lora.score' | 'harness.propose';
  payload: unknown; inverse: unknown };
```

Appendix B Glossary

Term	Meaning in this document
Worker / senior	The small resident reader (dense ~9B) and the larger on-demand reader (MoE, ~3B active).
Gate	The decision, per subtask, between the fast (habit) path and the slow (thinking) path.
Cascade	The ordered verifier: lint → type-check → unit tests → build → browser smoke; plus citation checks for prose.
Notebook	ACE-style explicit memory: facts, lessons, playbook entries, linked and delta-updated.
Skill	A verified, parameterized, tested piece of code with a contract and a success record.
Core block	The 1–2k-token slice of the Notebook present in every prompt.
Sleep	Idle-time consolidation: replay, merge, decay, promote, flag, re-index, pre-think, train.
LoRA pocket	A per-domain low-rank adapter trained weekly on verified work; archived with a score; never merged into the base.
Expert iteration	Generate → verify → train on what survived (STaR, ReST-EM, Absolute Zero).
Held-out set	Ten benchmark tasks never used for training or tuning, rotated monthly.
MoE	Mixture of experts: a model that activates a few expert sub-networks per token; saves compute, not memory.
QLoRA / DPO / SFT	Low-rank fine-tuning on a 4-bit base; preference optimization on better-vs-worse pairs; supervised fine-tuning.